Chapter 9
Analysis of Network Flow Data

9.1 Introduction

Many networks serve as conduits—either literally or figuratively—for *flows*, in the sense that they facilitate the movement of something, such as materials, people, or information. For example, transportation networks (e.g., of highways, railways, and airlines) support flows of commodities and people, communication networks allow for the flow of data, and networks of trade relations among nations reflect the flow of capital. We will generically refer to that of which a flow consists as *traffic*.

Flows are at the heart of the form and function of many networks, and understanding their behavior is often a goal of primary interest. Much of the quantitative work in the literature on flows is concerned with various types of problems involving, for example, questions of network design, provisioning, and routing, and their solutions have involved primarily tools in optimization and algorithms. Questions that are of a more statistical nature typically involve the modeling and prediction of network flow volumes, based on relevant data, and it is upon these topics that we will focus here.

Let $G = (V, E)$ be a network graph. Since flows have direction, from an origin to a destination, formally G will be a digraph. Following the convention in this literature, we will refer to the (directed) edges in this graph as links. Traffic typically passes over multiple links in moving between origin and destination vertices. A quantity of fundamental interest in the study of network flows is the so-called *origin-destination* (OD) matrix, which we will denote by $\mathbf{Z} = [Z_{ij}]$, where Z_{ij} is the total volume of traffic flowing from an origin vertex i to a destination vertex j in a given period of time. The matrix $\mathbf{Z}$ is also sometimes referred to as the *traffic matrix*.

In this chapter we will examine two statistical problems that arise in the context of network flows, distinguished by whether the traffic matrix $\mathbf{Z}$ is observed or to be predicted. More specifically, in Sect. 9.2, we consider the case where it is possible to observe the entire traffic matrix $\mathbf{Z}$ and it is of interest to model these observations. Modeling might be of interest both to obtain an understanding as to how potentially relevant factors (e.g., costs) affect flow volumes and to be able to make predictions

E.D. Kolaczyk and G. Csárdi, *Statistical Analysis of Network Data with R*, Use R! 65,
DOI 10.1007/978-1-4939-0983-4_9,

of future flow volumes. A class of models commonly used for such purposes are the so-called gravity models. In Sect. 9.3, we consider the problem of traffic matrix estimation. Here the case is assumed to be such that it is difficult or impossible to observe the traffic matrix entries Z_{ij} directly. Nevertheless, given sufficient information on marginal flow volumes (i.e., in the form of total volumes through vertices or over links) and on the routing of flows between origins and destinations, it is often possible to generate accurate predictions of $\mathbf{Z}$ using statistical methods.

9.2 Modeling Network Flows: Gravity Models

Gravity models are a class of models, developed largely in the social sciences, for describing aggregate levels of interaction among the people of different populations. They have traditionally been used most in areas like geography, economics, and sociology, for example, but also have found application in other areas of the sciences, such as hydrology and the analysis of computer network traffic. Our interest in gravity models in this chapter will be for their use in contexts where the relevant traffic flows are over a network of sorts.

The term 'gravity model' derives from the fact that, in analogy to Newton's law of universal gravitation, it is assumed that the interaction among two populations varies in direct proportion to their size, and inversely, with some measure of their separation. The concept goes back at least to the work of Carey [25] in the 1850's, but arguably was formulated in the strictest sense of the analogy by Stewart [136] in 1941, and has since been developed substantially in the past 50 years. Here we focus on a certain general version of the gravity model and corresponding methods of inference.

9.2.1 Model Specification

We let $\mathscr{I}$ and $\mathscr{J}$ represent sets of origins i and destinations j in V, of cardinality $I = |\mathscr{I}|$ and $J = |\mathscr{J}|$, respectively. Furthermore, Z_{ij} denotes—as defined in the introduction—a measure of the traffic flowing from $i \in \mathscr{I}$ to $j \in \mathscr{J}$ over a given period of time. Suppose that we are able to observe $\mathbf{Z} = [Z_{ij}]$ in full.

For example, the data set `calldata` contains a set of data for phone traffic between 32 telecommunication districts in Austria throughout a period during the year 1991.

```
#9.1 1 > library(sand)
     2 > data(calldata)
     3 > names(calldata)
     4 [1] "Orig"    "Dest"    "DistEuc" "DistRd"  "O.GRP"
     5 [6] "D.GRP"   "Flow"
```

These data, first described[1] by Fischer and Gopal [57], consist of $32 \times 31 = 992$ flow measurements z_{ij}, $i \neq j = 1, \ldots, 32$, capturing contact intensity over the period studied. In addition to basic labels of origin and destination region for each flow, there is information on the gross regional product (GRP) for each region, which may serve as a proxy for economic activity and income, both of which are relevant to business and private phone calls. Finally, there are two sets of measurements reflecting notions of distance between regions, one based on Euclidean distance and the other road-based.

As the original data in the variable `Flow` are in units of erlang (i.e., number of phone calls, including faxes, times the average length of the call divided by the duration of the measurement period), and the gravity modeling frameworks we will describe assume units of counts, we convert these data to quasi-counts, for the purposes of illustration.

```
#9.2 1 > min.call <- min(calldata$Flow)
     2 > calldata$FlowCnt <- round(5 * calldata$Flow / min.call)
```

These flow counts may be thought of as corresponding to (directed) edge weights in a graph, where the vertices represent telecommunication districts.

```
#9.3 1 > W <- xtabs(FlowCnt ~ Orig + Dest, calldata)
     2 > g.cd <- graph.adjacency(W, weighted=TRUE)
```

A plot of the resulting network graph is shown in Fig. 9.1.

```
#9.4 1 > in.flow <- graph.strength(g.cd, mode="in")
     2 > out.flow <- graph.strength(g.cd, mode="out")
     3 > vsize <- sqrt(in.flow + out.flow) / 100
     4 > pie.vals <- lapply((1:vcount(g.cd)),
     5 +   function(i) c(in.flow[i], out.flow[i]))
     6 > ewidth <- E(g.cd)$weight / 10^5
     7 > plot(g.cd, vertex.size=vsize, vertex.shape="pie",
     8 +   vertex.pie=pie.vals, edge.width=ewidth,
     9 +   edge.arrow.size=0.1)
```

Gravity models have been found to be useful in modeling such data. The *general gravity model* specifies that the traffic flows Z_{ij} be in the form of counts, with independent Poisson distributions and mean functions of the form

$$\mathbb{E}(Z_{ij}) = h_O(i)\, h_D(j)\, h_S(\mathbf{c}_{ij}), \tag{9.1}$$

where h_O, h_D, and h_S are positive functions, respectively, of the origin i, the destination j, and a vector $\mathbf{c}_{ij}$ of K so-called separation attributes. On a logarithmic scale, we therefore have

$$\log \mathbb{E}(Z_{ij}) = \log h_O(i) + \log h_D(j) + \log h_S(\mathbf{c}_{ij}). \tag{9.2}$$

[1] Data were originally collected by Manfred Fischer and Petra Staufer.

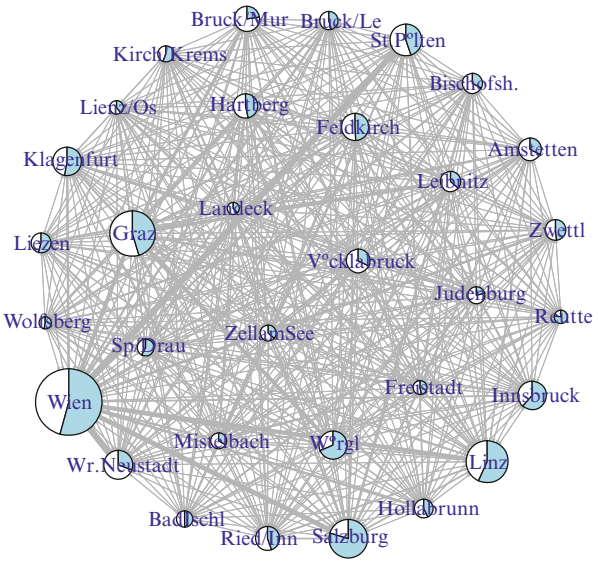

Fig. 9.1 Network representation of Austrian call network data flows in `calldata`. Vertex area reflects total flow volume in or out of a telecommunication region, with relative proportion of in- and out-flows indicated in *white* and *blue*, respectively. Width of links reflects volume of flow from origin to destination

As we shall see momentarily, common specifications of the functions h_O, h_D, and h_S tend to lead to expressions on the right-hand side of (9.2) that are linear in some unknown parameters. This log-linear form in turn facilitates the use of log-linear methods for statistical inference on the model parameters, as we discuss below in Sect. 9.2.2.

In general, the K elements of $\mathbf{c}_{ij}$ are chosen to quantify some notion(s) of separation ascribed to the origin-destination pair (i, j), often based on concepts of 'distance' or 'cost' of either a literal or figurative nature. The functions h_O and h_D are sometimes referred to as the *origin* and *destination functions*, respectively, while h_S is commonly called the *separation* or *deterrence function*. Often the separation function is constrained to be non-increasing in the elements of $\mathbf{c}_{ij}$.

An early and now classical example of the gravity model is that of Stewart [136], proposed in connection with his theory of 'demographic gravitation,' which specifies that

$$\mathbb{E}(Z_{ij}) = \gamma \, \pi_{O,i} \, \pi_{D,j} \, d_{ij}^{-2}, \tag{9.3}$$

where $\pi_{O,i}$ and $\pi_{D,j}$ are measures of the origin and destination population sizes, respectively, for two geographical regions i and j, and d_{ij} is a measure of distance between the centers of these regions. This formulation is completely analogous to Newton's universal law, right down to the use of a 'demographic gravitational constant' γ. However, unlike Newton's law, neither empirical evidence nor theoretical arguments suggest that this form of the gravity model is strictly accurate in practical contexts, where somewhat more flexible forms are used.

For example, consider again the Austrian call data. The gross regional products `O.GRP` and `D.GRP` can be used in analogy to population size, and we use the road-based measure of distance `DistRd`. A comparison of these variables against flow volume, on a logarithmic scale, is shown in Fig. 9.2.

```
#9.5 1 > calldata$lFlowCnt <- log(calldata$FlowCnt, 10)
     2 > calldata$lO.GRP <- log(calldata$O.GRP, 10)
     3 > calldata$lD.GRP <- log(calldata$D.GRP, 10)
     4 > calldata$lDistRd <- log(calldata$DistRd, 10)
     5 >
     6 > library(car)
     7 > scatterplotMatrix( ~ lFlowCnt + lO.GRP + lD.GRP +
     8 +   lDistRd, data=calldata)
```

It is evident from examination of the scatterplots in this figure that there is a reasonably strong relationship between call volume and the origin GRP, destination GRP, and distance. Moreover, this relationship is fairly linear (i.e., the green lines, produced by ordinary least-squares, lie close to the solid red lines, which are the result of a nonparametric smoother) and is increasing in origin and destination GRP and decreasing in distance. These observations suggest that a model of the form

$$\mathbb{E}(Z_{ij}) = \gamma\,(\pi_{O,i})^{\alpha}\,(\pi_{D,j})^{\beta}\,(c_{ij})^{\theta} \tag{9.4}$$

might be reasonable, where $\pi_{O,i}$ is the GRP of origin i, $\pi_{D,j}$ is the GRP of destination j, and c_{ij} is the distance from origin i to destination j. That is, a simple loglinear model of the form

```
#9.6 1 > formula.s <- FlowCnt ~ lO.GRP + lD.GRP + lDistRd
```

seems sensible. This choice corresponds to origin and destination functions of the form

$$h_O(i) = (\pi_{O,i})^{\alpha} \quad \text{and} \quad h_D(j) = (\pi_{D,j})^{\beta}\,, \tag{9.5}$$

and separation function,

$$h_S(c_{ij}) = (c_{ij})^{\theta} \tag{9.6}$$

in the general gravity model.

Alternatively, the values $\{h_O(i)\}_{i\in\mathscr{I}}$ and $\{h_D(j)\}_{j\in\mathscr{J}}$ are often simply treated as a collection of $I+J$ unknown parameters. For instance, adopting this specification for the Austrian call data, while maintaining the same choice of separation function, yields the following more general formula.

```
#9.7 1 > formula.g <- FlowCnt ~ Orig + Dest + lDistRd
```

For a comprehensive treatment of additional variations on the general gravity model, including versions that relax the assumption of independence between origin and destination effects implicit in the product form $h_O(i) \times h_D(j)$, see Sen and Smith [130].

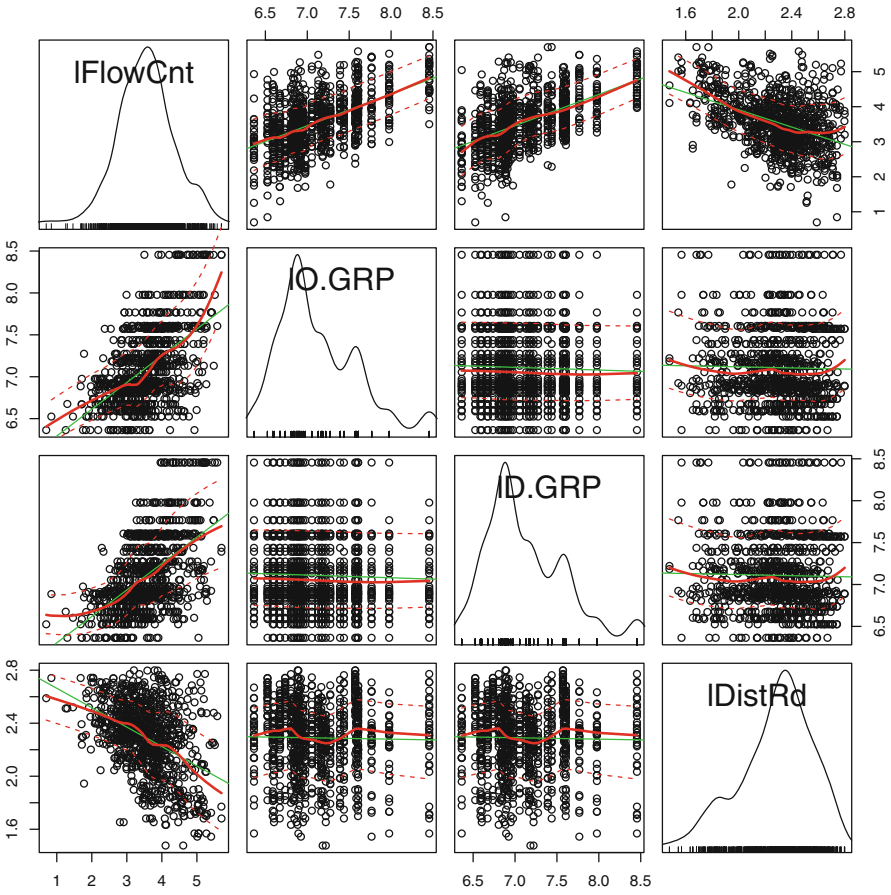

Fig. 9.2 Austrian call data. Scatterplots are shown for call flow volume versus each of origin GRP, destination GRP, and distance, along the top row, and for the latter three variables against each other, in the other rows. All axes are on log-log scales. Superimposed on each scatterplot are two *solid lines*, for descriptive purposes, showing fits based on simple linear regression (*green*) and a nonparametric smoother (*solid red*). Density plots for each of the four variables are shown along the diagonal

9.2.2 Inference for Gravity Models

In light of the specification that the Z_{ij} be independent Poisson random variables with means $\propto_{ij} = \mathbb{E}(Z_{ij})$, statistical inference in the general gravity model is most naturally approached through likelihood-based methods. Moreover, as remarked already, typical specifications of the general gravity model are in fact log-linear models, which in turn are a specific instance of the class of generalized linear models. See, for example, McCullagh and Nelder [107, Ch. 6]. As a result, these models often can be fit using standard software.

To be concrete, consider the model underlying `formula.g` above. This model can be expressed in the form

$$\log \propto_{ij} = \alpha_i + \beta_j + \theta^T \mathbf{c}_{ij}, \tag{9.7}$$

where $\alpha_i = \log h_O(i)$, $\beta_j = \log h_D(j)$, and $\theta, \mathbf{c}_{ij}$ are vectors of length K (with $K = 1$ in the context of our call data example). Other cases, such as when the origin and destination functions are parameterized, as in the expression (9.4) underlying `formula.s` above, may be handled similarly.

Let $\mathbf{Z} = \mathbf{z}$ be an $(IJ) \times 1$ vector of observations of the flows Z_{ij}, which for convenience are usually ordered by origin i, and by destination j within origin i (as is the case with the data set `calldata`). The relevant portion of the Poisson log-likelihood for $\propto$ takes the form

$$\ell(\propto) = \sum_{i,j \in \mathscr{I} \times \mathscr{J}} z_{ij} \log \propto_{ij} - \propto_{ij}. \tag{9.8}$$

Substituting the gravity model (9.7), taking partial derivatives with respect to the parameters α_i, β_j, and θ_k, setting the resulting equations equal to zero, and simplifying, it can be shown that the maximum likelihood estimates for these parameters must yield estimates $\hat{\propto}_{ij} = \hat{\alpha}_i \hat{\beta}_j \exp(\hat{\theta}^T \mathbf{c}_{ij})$, for $\propto_{ij}$ satisfying the equations

$$\hat{\propto}_{i+} = z_{i+}, \text{ for } i \in \mathscr{I} \quad \text{and} \quad \hat{\propto}_{+j} = z_{+j}, \text{ for } j \in \mathscr{J} \tag{9.9}$$

$$\sum_{i,j \in \mathscr{I} \times \mathscr{J}} c_{ij;k} \hat{\propto}_{ij} = \sum_{i,j \in \mathscr{I} \times \mathscr{J}} c_{ij;k} z_{ij}, \text{ for } k = 1, \ldots, K, \tag{9.10}$$

where $\hat{\propto}_{i+} = \sum_{j \in \mathscr{J}} \propto_{ij}$ and $\hat{\propto}_{+j} = \sum_{i \in \mathscr{I}} \propto_{ij}$, and the z_{i+} and z_{+j} are defined similarly. Here $c_{ij;k}$ denotes the kth element of $\mathbf{c}_{ij}$.

Under mild conditions, these estimates will be well defined, and furthermore, the values $\hat{\theta}_k$ and $\hat{\propto}_{ij}$ will be unique. The values $\hat{\alpha}_i$ and $\hat{\beta}_j$ will be unique only up to a constant, due to the fact that the underlying model is over-parameterized by one degree of freedom.[2] Note that we are assuming here, without loss of generality, that origins i and destinations j for which $z_{i+} = 0$ or $z_{+j} = 0$ are dropped, as they contribute nothing to the analysis.

Various algorithms may be used to calculate the maximum likelihood estimates. Most straightforward is to use standard software for fitting log-linear models, usually available as an option in routines for fitting generalized linear models. These procedures are based on an iteratively re-weighted least-squares algorithm, like that used in logistic regression, which derives from application of the

[2] More precisely, we can write (9.7) in the form $\log(\propto) = \mathbf{M}\gamma$, where $\mathbf{M}$ is an $(IJ) \times (I + J + K)$ matrix, and $\gamma = (\alpha_1, \ldots, \alpha_I, \beta_1, \ldots, \beta_J, \theta_1, \ldots, \theta_K)^T$ is an $(I + J + K) \times 1$ vector. The first $I + J$ columns of $\mathbf{M}$ are binary vectors, indicating the appropriate origin and destination for each entry of $\propto$, and are redundant in that both the first I and the next J sum to the unit vector. The last K columns correspond to the K variables defining the $\mathbf{c}_{ij}$. Assuming that the latter are linearly independent of themselves and of the former, the rank of $\mathbf{M}$ will be $(I + J - 1) + K$. See Sen and Smith [130, Chap. 5.2].

Newton-Raphson algorithm. See, for example, McCullagh and Nelder [107, Chap. 2.5] for details. Standard output includes parameter estimates and approximate standard errors, where the latter are driven by the usual arguments for asymptotic normality of maximum likelihood estimators.

To illustrate, we use the function `glm` in the R base package to fit our simple (i.e,. `formula.s`) and more general (i.e., `formula.g`) specifications of gravity models for the Austrian call data.

```
#9.8 1 > gm.s <- glm(formula.s, family="poisson", data=calldata)
     2 > gm.g <- glm(formula.g, family="poisson", data=calldata)
```

The results of the fitted model `gm.s` are as follows.

```
#9.9 1 > summary(gm.s)
     2
     3 Call:
     4 glm(formula=formula.s, family="poisson", data=calldata)
     5
     6 Deviance Residuals:
     7     Min       1Q   Median       3Q      Max
     8 -475.06   -54.16   -29.20    -2.09  1149.93
     9
    10 Coefficients:
    11               Estimate Std. Error z value Pr(>|z|)
    12 (Intercept) -1.149e+01  5.394e-03   -2131   <2e-16 ***
    13 lO.GRP       1.885e+00  4.306e-04    4376   <2e-16 ***
    14 lD.GRP       1.670e+00  4.401e-04    3794   <2e-16 ***
    15 lDistRd     -2.191e+00  7.909e-04   -2770   <2e-16 ***
    16 ---
    17 Signif. codes:  0 *** 0.001 ** 0.01 * 0.05 . 0.1  1
    18
    19 (Dispersion parameter for poisson family taken to be 1)
    20
    21     Null deviance: 45490237  on 991  degrees of freedom
    22 Residual deviance: 10260808  on 988  degrees of freedom
    23 AIC: 10270760
    24
    25 Number of Fisher Scoring iterations: 5
```

We see that effects due to origin, destination, and distance are all found to be highly significant, as would be expected from examination of the scatterplots in Fig. 9.2.

Nevertheless, a comparison of the Akaike information criterion (AIC) statistic[3] for the two models

[3] The AIC statistic for a likelihood-based model, with k-dimensional parameter η, is defined as $AIC = -2\ell(\hat{\eta}) + 2k$, where $\ell(\eta)$ is the log-likelihood evaluated at η, and $\hat{\eta}$ is the maximum likelihood estimate of η. This statistic, as with others of its type, provides an estimate of the generalization error associated with the fitted model, in this case effectively by off-setting the assessment of how well the model fits the data by a measure of its complexity. See, for example, Hastie, Tibshirani, and Friedman [71, Chap. 7.5] for additional details.

```
#9.10 1 > gm.g$aic
      2 [1] 5466814
      3 > gm.s$aic
      4 [1] 10270760
```

indicates that the more general model is vastly better than the simpler model, without necessarily overfitting, despite the fact that the former incorporates 64 variables, compared to four variables in the latter.

A comparison of the fitted values $\hat{\alpha}_{ij}$, from the model `gm.g`, against the observed flow volumes z_{ij} is shown in Fig. 9.3.

```
#9.11 1 > plot(calldata$lFlowCnt, log(gm.g$fitted.values,10),
      2 +   cex.lab=1.5,
      3 +   xlab=expression(Log[10](paste("Flow Volume"))),
      4 +   col="green", cex.axis=1.5, ylab="", ylim=c(2, 5.75))
      5 > mtext(expression(Log[10](paste("Fitted Value"))), 2,
      6 +   outer=T, cex=1.5, padj=1)
      7 > abline(0, 1, lwd=2, col="darkgoldenrod1")
```

This comparison is shown on a log-log scale, due to the large dynamic range of the values involved. The relationship between the two quantities is found to be fairly linear, although arguably a bit better for medium- and large-volume flows than for low-volume flows.

Also shown in Fig. 9.3 is a comparison of the relative errors $(z_{ij} - \hat{\alpha}_{ij})/z_{ij}$ against the flow volumes z_{ij}. The comparison is again on a log-log scale. For the relative errors, the logarithm is applied to the absolute value, and then the sign of the error is reintroduced through the use of two shades of color.

```
#9.12 1 > res <- residuals.glm(gm.g, type="response")
      2 > relres <- res/calldata$FlowCnt
      3 > lrelres <- log(abs(relres), 10)
```

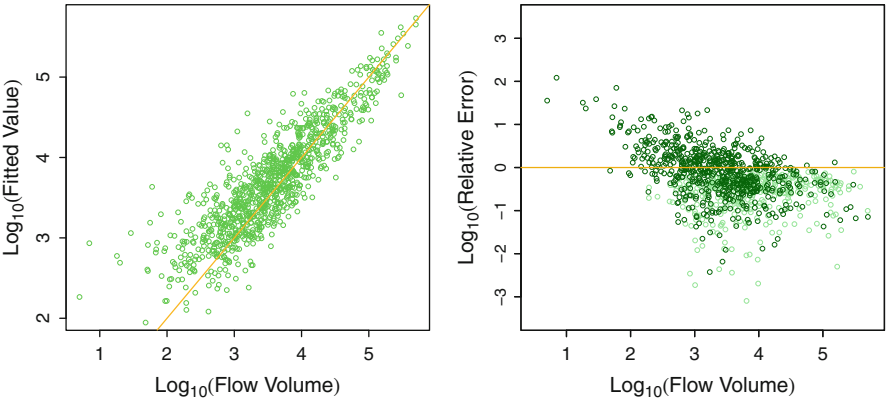

Fig. 9.3 Accuracy of estimates of traffic volume made by the gravity models for the Austrian call data. *Left*: Fitted values versus flow volume. *Right*: Relative error versus flow volume, where *light* and *dark* points indicate under- and over-estimation, respectively. All axes are on logarithmic scales, base ten. The lines $y = x$ and $y = 0$ are shown in *yellow* in the *left* and *right* plots, respectively, for reference

```
> res.sgn <- (relres>=0)
>
> plot(calldata$lFlowCnt[res.sgn], lrelres[res.sgn],
+   xlim=c(0.5, 5.75), ylim=c(-3.5, 3.5),
+   xlab=expression(Log[10](paste("Flow Volume"))),
+   cex.lab=1.5, cex.axis=1.5, ylab="", col="lightgreen")
> mtext(expression(Log[10](paste("Relative Error"))), 2,
+   outer=T, cex=1.5, padj=1)
> par(new=T)
> plot(calldata$lFlowCnt[!res.sgn], lrelres[!res.sgn],
+   xlim=c(0.5, 5.75), ylim=c(-3.5, 3.5),
+   xlab=expression(Log[10](paste("Flow Volume"))),
+   cex.lab=1.5, cex.axis=1.5, ylab="", col="darkgreen")
> mtext(expression(Log[10](paste("Relative Error"))), 2,
+   outer=T, cex=1.5, padj=1)
> abline(h=0, lwd=2, col="darkgoldenrod2")
```

We see that the relative error varies widely in magnitude. A large proportion of the flows are estimated with an error on the order of z_{ij} or less (i.e., the logarithm of relative error is less than zero), but a substantial number are estimated with an error on the order of up to ten times z_{ij}, and a few others are even worse. In addition, we can see that, roughly speaking, the relative error decreases with volume. Finally, it is clear that for low volumes the model is inclined to over-estimate, while for higher volumes, it is increasingly inclined to under-estimate.

9.3 Predicting Network Flows: Traffic Matrix Estimation

In many types of networks, it is difficult—if not effectively impossible—to actually measure the flow volumes Z_{ij} on a network. Nevertheless, knowledge of traffic flow volumes is fundamental to a variety of network-oriented tasks, such as traffic management, network provisioning, and planning for network growth. Fortunately, in many of the same contexts in which measurement of the flow volumes Z_{ij} between origins and destinations is difficult, it is often relatively easy to instead measure the flow volumes on network links. For example, in highway road networks, sensors may be positioned at the entrances to on- and off-ramps. Similarly, routers in an Internet network come equipped with the facility to monitor the data on incident links. Measurements of flow volumes on network links, in conjunction with knowledge of the manner in which traffic flows over these links, between origins and destinations, can be sufficient to allow us to accurately predict origin-destination volumes. This is known as the *traffic matrix estimation* problem.

Formally, let X_e denote the total flow over a given link $e \in E$, and let $\mathbf{X} = (X_e)_{e \in E}$. The link totals in $\mathbf{X}$ can be related to the origin-destination flow volumes in $\mathbf{Z}$ through the expression $\mathbf{X} = \mathbf{BZ}$, where $\mathbf{Z}$ now represents our traffic matrix written as a vector and $\mathbf{B}$ is the so-called *routing matrix*, describing the manner in which traffic moves throughout the network. The traffic matrix estimation problem then refers to

predicting the Z_{ij} from the observed link counts $\mathbf{X} = (X_e)_{e \in E}$. Here we will focus on the case that each origin-destination pair (i, j) has only a single route between them, in which case $\mathbf{B}$ is a binary matrix,[4] with the entry in the row corresponding to link e and the column corresponding to pair (i, j) being

$$B_{e;ij} = \begin{cases} 1, & \text{if link } e \text{ is traversed in going from } i \text{ to } j \ , \\ 0, & \text{otherwise} \ . \end{cases} \tag{9.11}$$

In any case of practical interest, the traffic matrix estimation problem will be highly under-constrained, in the sense that we effectively seek to invert the routing matrix $\mathbf{B}$ in the relation $\mathbf{X} \approx \mathbf{BZ}$, and $\mathbf{B}$ typically has many fewer rows (i.e., network links) than columns (i.e., origin-destination pairs). Various additional sources of information therefore typically are incorporated into the problem, which effectively serve to better constrain the set of possible solutions. Methods proposed in this area can be roughly categorized as *static* or *dynamic*, depending on whether they are aimed at estimating a traffic matrix for a single time period or successively over multiple time periods. In this section we will content ourselves, after a brief illustration of the nature of the traffic estimation problem, with introducing just one static method—the so-called tomogravity method—which has had a good deal of success in the context of the Internet.

9.3.1 An Ill-Posed Inverse Problem

The various static methods proposed for traffic matrix estimation are similar in that they all ultimately involve the optimization of some objective function, usually subject to certain constraints. However, they can differ widely in the construction of and justification for this objective function and, to a lesser extent, the constraints imposed.

One common approach is through the use of least-squares principles, which can be motivated by a Gaussian measurement model. This model specifies that

$$\mathbf{X} = \mathbf{B} \propto + \varepsilon, \tag{9.12}$$

where $\mathbf{X} = (X_e)_{e \in E}$ and $\mathbf{B}$ are defined as above, $\propto$ is an $IJ \times 1$ vector of expected flow volumes over all origin destination pairs, and ε is an $N_e \times 1$ vector of errors. In principle, this formulation suggests that $\propto$ be estimated through ordinary least squares, i.e., by minimizing

$$(\mathbf{x} - \mathbf{B} \propto)^T (\mathbf{x} - \mathbf{B} \propto), \tag{9.13}$$

[4] If multiple routes are possible, the entries of $\mathbf{B}$ are instead fractions representing, for example, the proportion of traffic from i to j that is expected to use the link e.

where **x** is the vector of observed link flow volumes. However, typically N_e is much smaller than IJ, and so the least-squares problem is under-determined and there will in general be an infinite number of possible solutions $\hat{\propto}$. That is, this traffic matrix estimation problem is ill-posed.

To illustrate, we will use the data set `bell.labs` in the **networkTomography** package.

```
#9.13 1 > library(networkTomography)
      2 > data(bell.labs)
```

These data correspond to measurements of traffic flow volume on a small computer network at Bell Laboratories, Lucent Technologies, as reported by Cao et al. [23]. The network consisted of four computing devices—named *fddi*, *local*, *switch*, and *corp*—connected to a single router. A representation of this network is shown in Fig. 9.4.

```
#9.14 1 > g.bl <- graph.formula(fddi:switch:local:corp ++ Router)
      2 > plot(g.bl)
```

Renaming some of the variables in this data set in a manner consistent with the notation of this chapter,

```
#9.15 1 > B <- bell.labs$A
      2 > Z <- bell.labs$X
      3 > x <- bell.labs$Y
```

we see that we have available to us not only the link flow volumes **X** and the routing matrix **B**, but also the actual origin-destination flow volumes **Z**, the latter which are useful for purposes of validation. The flow volumes in **X** and **Z** are in units of average bytes per five-minute interval. The data were collected continuously over a twenty-four period.

Ignoring the single router in this network, which simply passes on traffic it receives, rather than itself generating traffic, there are a total of four vertices in this network with a total of eight links (i.e., one in-going and out-going link per vertex). Hence, we have 8 link-level traffic time series. Using the **lattice** package, and working from the data frame version of the underlying network and measurements encoded in `bell.labs$df`, we can produce a useful visualization that shows all eight of these time series simultaneously.

```
#9.16 1 > library(lattice)
      2 > traffic.in <- c("dst fddi", "dst switch",
      3 +   "dst local", "dst corp")
      4 > traffic.out <- c("src fddi", "src switch",
      5 +   "src local", "src corp")
      6 > my.df <- bell.labs$df
      7 > my.df$t <- unlist(lapply(my.df$time, function(x) {
      8 +     hrs <- as.numeric(substring(x, 11, 12))
      9 +     mins <- as.numeric(substring(x, 14, 15))
     10 +     t <- hrs + mins/60
     11 +     return(t)}))
     12 >
```

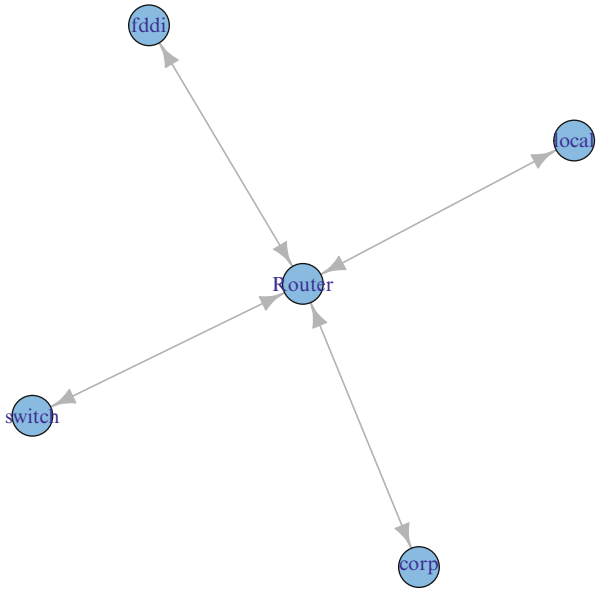

Fig. 9.4 Schematic representation of the small computer network underlying the data in `bell.labs`

```
> # Separate according to whether data
> # are incoming or outgoing.
> my.df.in <- subset(my.df, nme %in% traffic.in)
> my.df.out <- subset(my.df, nme %in% traffic.out)

>
> # Set up trellis plots for each case.
> p.in <- xyplot(value / 2^10 ~ t | nme, data=my.df.in,
+   type="l", col.line="goldenrod",
+   lwd=2, layout=c(1, 4),
+   xlab="Hour of Day", ylab="Kbytes/sec")
> p.out <- xyplot(value / 2^10 ~ t | nme, data=my.df.out,
+   type="l", col.line="red",
+   lwd=2, layout=c(1, 4),
+   xlab="Hour of Day", ylab="Kbytes/sec")
>
> # Generate trellis plots.
> print(p.in, position=c(0, 0.5, 1, 1), more=TRUE)
> print(p.out, position=c(0, 0, 1, 0.5))
```

The resulting plots are shown in Fig. 9.5. It is evident, looking at some of the more pronounced features of the individual curves, how the traffic from one source (e.g., *corp*, around hour 12) contributes to the traffic to another destination (e.g., *switch*, at the same time). The goal of traffic matrix estimation is to extract these underlying individual contributions from the aggregate link-level measurements shown in the figure.

Now let us examine the routing matrix for this network. The matrix `B` comes to us expressed in a full-rank form, by having dropped one row from the overall routing matrix. For the purposes of illustration, we augment this matrix by adding back this missing row (yielding a matrix of reduced rank, but not changing the overall problem).

```
#9.17 1 > B.full <- rbind(B, 2 - colSums(B))
      2 > write.table(format(B.full),
      3 +   row.names=F, col.names=F, quote=F)
      4 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0
      5 0 0 0 0 1 1 1 1 0 0 0 0 0 0 0 0
      6 0 0 0 0 0 0 0 0 1 1 1 1 0 0 0 0
      7 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1
      8 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0
```

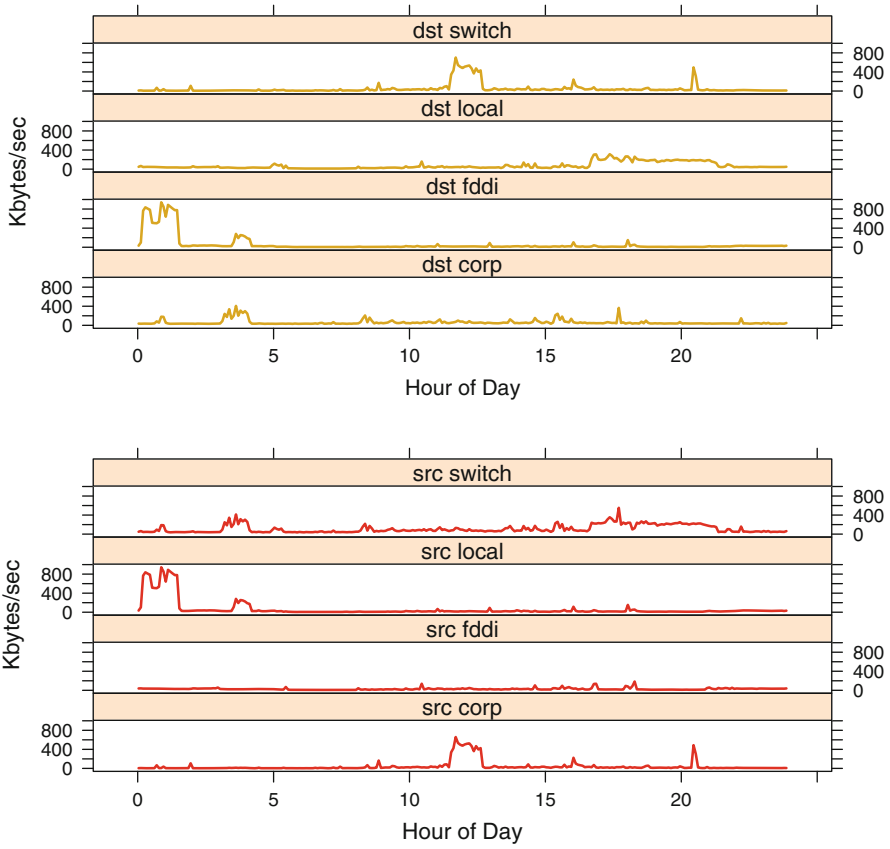

Fig. 9.5 Time series showing traffic volume over the eight links in the Bell Labs network. *Top*: Traffic by destination. *Bottom*: Traffic by source

```
 9 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0
10 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0
11 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1
```

This matrix has sixteen columns, corresponding to each origin-destination pair (including the four cases where a machine sends traffic to itself). Similarly, it has eight rows, corresponding to the four machines *fddi*, *local*, *switch*, and *corp*, respectively, first as origins and then as destinations. It is straightforward to derive this matrix from the topology of the network shown in Fig. 9.4. For example, the second column corresponds to the flow from *fddi* to *local*, while the seventh column corresponds to the flow from *local* to *switch*.

Clearly for this network the least squares problem in (9.13) is under-determined, in seeking to recover information on sixteen origin-destination flows from only eight link-level flows.

9.3.2 The Tomogravity Method

A standard approach to tackling ill-posed problems like that in (9.13) is to augment the objective function with some sort of penalty (also referred to as regularization), so as to restrict the collection of possible solutions in a useful manner. The *tomogravity method* is in this spirit, for the specific problem of traffic matrix estimation. Introduced by Zhang, Roughan, Lund, and Donoho [151], the 'tomo' part of the name refers to the fact that the traffic matrix estimation problem is considered a form of 'network tomography' (we encountered another example in Chap. 7.4), a term coined by Vardi [142]. The 'gravity' part of the name, in turn, refers to the fact that a simple gravity model is used to constrain the solutions produced by this method.

In the tomogravity method, we replace the least-squares criterion in (9.13) by a penalized least-squares criterion of the form

$$(\mathbf{x} - \mathbf{B}\propto)^T(\mathbf{x} - \mathbf{B}\propto) + \lambda D\left(\propto || \propto^{(0)}\right), \tag{9.14}$$

where again the goal is to minimize the overall objective function. Here the penalty

$$D(\propto || \propto^{(0)}) = \sum_{ij} \frac{\propto_{ij}}{\propto_{++}} \log \frac{\propto_{ij}}{\propto_{ij}^{(0)}} \tag{9.15}$$

is the relative entropy 'distance' between a candidate solution $\propto$ and some pre-specified vector $\propto^{(0)}$. This quantity is also known as the Kullback–Liebler divergence and summarizes how similar the two 'distributions' $\{\propto_{ij}/\propto_{++}\}$ and $\{\propto_{ij}^{(0)}/\propto_{++}^{(0)}\}$ are to each other, where $\propto_{++}$ and $\propto_{++}^{(0)}$ are the sum of the elements in $\propto$ and $\propto^{(0)}$, respectively. It is always non-negative and will be equal to zero if and only if $\propto = \propto^{(0)}$.

The notion of a gravity model enters into the tomogravity method by specifying $\propto^{(0)}$ to have a certain multiplicative form reminiscent of a simple gravity model. Specifically, we let

$$\propto_{ij}^{(0)} = Z_{i+}^{(0)} \times Z_{+j}^{(0)} \tag{9.16}$$

be the product of the net out-flow and in-flow at vertices i and j, respectively. In addition, the constraint $\propto_{++} = Z_{++}^{(0)}$ is enforced, where $Z_{++}^{(0)}$ is the total traffic in the network. Importantly, these values all can be obtained through appropriate summation of the elements in the vector $\mathbf{x}$ of observed link counts.

Also necessary to completely specify the criterion in (9.14) is the value λ. This value acts as a smoothing parameter, with larger values encouraging solutions $\hat{\propto}$ that are closer to the pre-specified $\propto^{(0)}$. A value of $\lambda = (0.01)^2$ is report to work well in practice.

The regularized least-squares problem in (9.14) can be solved by methods of convex optimization. Note that $\hat{\propto}$, as an estimate of the expected flow volumes $\propto$, does not necessarily satisfy the constraint $\mathbf{x} = \mathbf{Bz}$. If it is desired to enforce this constraint (i.e., if the measured link volumes $\mathbf{x}$ are without error), then the iterative proportional fitting procedure (IPFP) may be used. IPFP, usually credited to Deming and Stephan [44] in the statistics literature, is a standard tool in the statistical analysis of contingency tables, where it is used to force the elements of a table to match the marginals. See [23, Sec 2.4] for a description of its use in the context of traffic matrix estimation.

The package **networkTomography** implements the tomogravity method, with IPFP adjustment, in the function `tomogravity`. We illustrate by applying it to the Bell Labs data.

```
#9.18 1 > x.full <- Z %*% t(B.full)
      2 > tomo.fit <- tomogravity(x.full, B.full, 0.01)
      3 > zhat <- tomo.fit$Xhat
```

The plotting capabilities of the **lattice** package are again useful, now for comparing the resulting predictions for the sixteen origin-destination pairs in this network against the actual origin-destination flows.

```
#9.19 1 > nt <- nrow(Z); nf <- ncol(Z)
      2 > t.dat <- data.frame(z = as.vector(c(Z) / 2^10),
      3 +   zhat = as.vector(c(zhat) / 2^10),
      4 +   t <- c(rep(as.vector(bell.labs$tvec), nf)))
      5 >
      6 > od.names <- c(rep("fddi->fddi", nt),
      7 +   rep("fddi->local", nt),
      8 +   rep("fddi->switch", nt), rep("fddi->corp", nt),
      9 +   rep("local->fddi", nt), rep("local->local", nt),
     10 +   rep("local->switch", nt), rep("local->corp", nt),
     11 +   rep("switch->fddi", nt), rep("switch->local", nt),
     12 +   rep("switch->switch", nt), rep("switch->corp", nt),
     13 +   rep("corp->fddi", nt), rep("corp->local", nt),
     14 +   rep("corp->switch", nt), rep("corp->corp", nt))
     15 >
```

```
> t.dat <- transform(t.dat, OD = od.names)
>
> xyplot(z~t | OD, data=t.dat,
+   panel=function(x, y, subscripts){
+     panel.xyplot(x, y, type="l", col.line="blue")
+     panel.xyplot(t.dat$t[subscripts],
+     t.dat$zhat[subscripts],
+     type="l", col.line="green")
+   }, as.table=T, subscripts=T, xlim=c(0, 24),
+   xlab="Hour of Day", ylab="Kbytes/sec")
```

The results are shown in Fig. 9.6 and indicate that the tomogravity method appears to do quite well in capturing the flow volumes across the network, despite the highly variable nature of these time series and the somewhat substantial dynamic range (i.e., almost one megabyte order of magnitude on the *y*-axis).

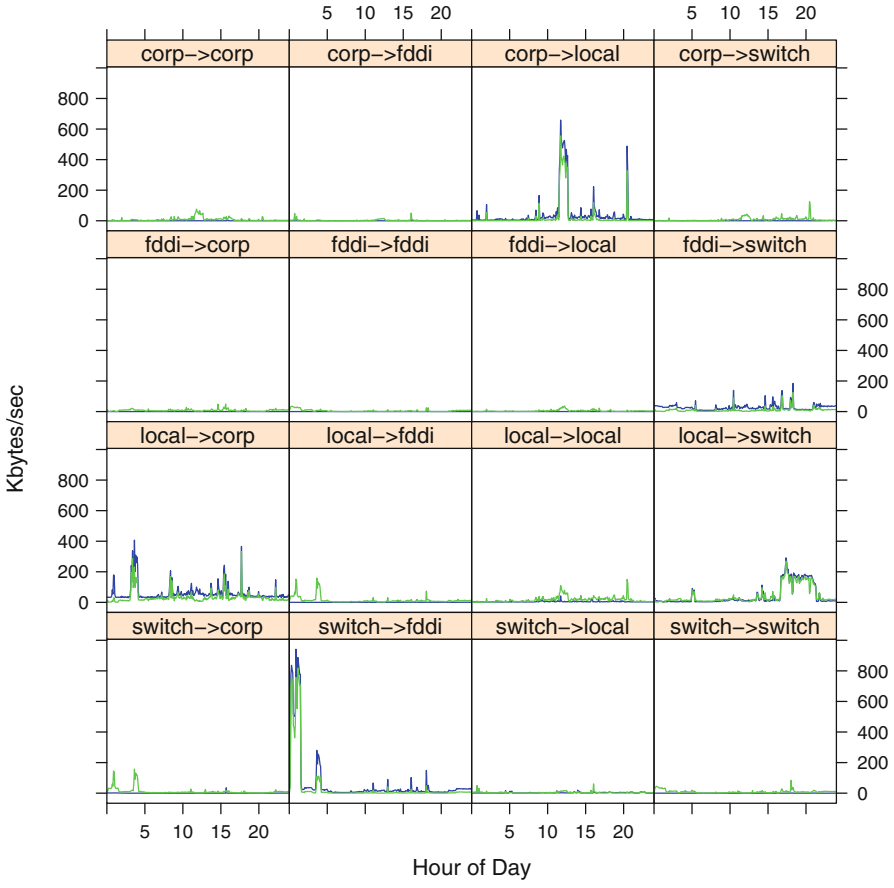

Fig. 9.6 Time series showing the prediction of the actual origin-destination flow volumes (*blue*) by the tomogravity method (*green*) for the Bell Labs network

9.4 Additional Reading

For a thorough introduction to the topic of gravity models, see Sen and Smith [130]. A review of the traffic matrix estimation problem, and other related problems in network tomography, may be found in Castro et al. [27].